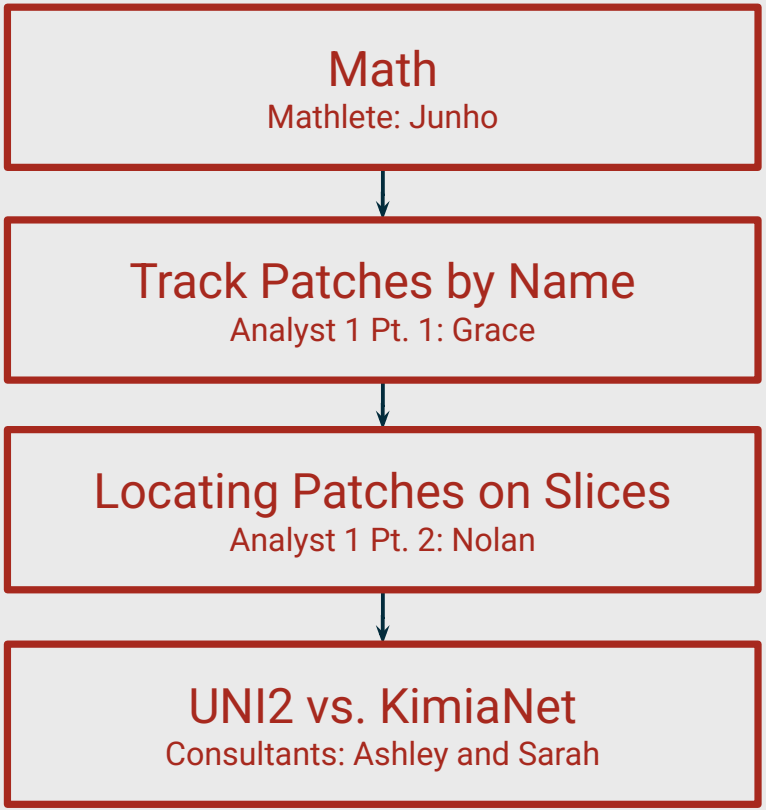
Ashley Cheng, Grace Fan, Junho Hong, Sarah Villamil, Nolan White

Presentation 2

Team1_FullStack

Overview


Math
Mathlete: Junho
Track Patches by Name
Analyst 1 Pt. 1: Grace
Locating Patches on Slices
Analyst 1 Pt. 2: Nolan
UNI2 vs. KimiaNet
Consultants: Ashley and Sarah

Math

Mathlete

Nested-CV search design

Within outer fold A, folds B–E tune the hyperparameters

	A	B	C	D	E
inner 1	unused	val	train	train	train
inner 2	unused	train	val	train	train
inner 3	unused	train	train	val	train
inner 4	unused	train	train	train	val

Training runs = 5 outer folds × N trials × K inner models

Each outer fold gets its own Optuna search, which never sees that fold. Each trial trains K = 4 inner models and is scored by their mean validation loss.

What to tune
5 of 18 tuned

Scales
log or linear

Objective
balanced CE

Budget
800 runs

Sampler
Sobol, then TPE

1. The old weight-decay range barely acts
2. The objective sets the class weight
3. Trials within the noise floor cannot be ranked
4. Default TPE trusts as few as 2 trials

Base code: Presentation2_base_code (optuna_training.py, trainer.py, models.py, config.py)

Hyperparameters to tune

Priority	Hyperparameter	Range	Scale
1	**Learning rate** Sets the size of every update	10^{-5} – 10^{-3}	log
2	**Weight decay** Acts on all 2.49 M weights directly	1% – 63% by epoch 4	log
3	**Classifier dropout** Perturbs the case vector directly	0 – 0.5	linear
4	**Entropy weight λ** Moves the loss by at most λ	10^{-3} – 1	log
5	**Projector dropout** Its noise is averaged over many patches	0 – 0.5	linear

Kept fixed

The patch cap never removes a patch, since the largest slice has 500.

The slice cap is removed, since it drops slices in 20 case–stain sets.

2 **The class weights** are set by the objective.

Image size and the 32 px minimum were fixed when the embeddings were computed.

Batch size 1 and no attention dropout are fixed by the code.

Attention and embedding sizes, entropy mode, scheduler, early stopping and epochs stay at the base values, as their expected effect is lower.

Base code: config.py L18–54 · trainer.py L260 · dataset.py L77–78, L113–114 · models.py L70–93 · precompute_pooled_features.py L32
Eighteen candidates from the assignment's list and Team 3's Presentation 1 list

Ranges and scales

Learning rate η log scale

Adam's step $|\Delta\theta| \lesssim \eta$

Each factor of η changes every step by the same ratio, so the scale is log. At 10^{-5} a weight moves at most about 0.017 in 30 epochs, twice the projector's initial spread, and at 10^{-3} the pilot's training loss jumped from 0.24 back to 3.0.

Weight decay λ_{wd} log scale

After T steps of decay $\theta \longrightarrow \theta \cdot (1 - \eta \cdot \lambda_{wd})^{T}$

1 The old range shrinks weights by at most 0.34% over a whole run.

I set the range by the shrink reached at epoch 4, the pilot's best epoch (224 steps = 56 cases × 4 epochs), from 1% to 63%.

Dropout p (two layers) linear scale

Var(noise) / signal² $= p / (1 - p)$

p = 0 is a valid choice, so the scale is linear. At p = 0.5 the noise already equals the signal, so the range stops there.

Entropy weight λ log scale

$\lambda \cdot H \leq \lambda, \quad H \in [0, 1]$

At λ = 10^{-3} the term was 0.05% of the first-epoch loss, and at λ = 1 it matches that loss. The useful λ spans three orders of magnitude, so the scale is log.

Base code: trainer.py L57–61, L195–205, L269 · models.py L70–93 · torch/optim/adam.py L411 (decay step)
Kingma & Ba 2015, Sec. 2.1 (|Δ| ≲ η) · Loshchilov & Hutter 2019 (decay and steps) · pilots at η = $5 \cdot 10^{-5}$, $2.08 \cdot 10^{-4}$, 10^{-3}

Objective and class weight

$$\textit{min over } q \quad \Sigma_y \; w_y \cdot p(y \mid x) \cdot (-\log q_y) \quad \Longrightarrow \quad q_y \propto w_y \cdot p(y \mid x)$$

w_y: class weight · $p(y \mid x)$: true class probability · q_y: model's prediction · solved with a Lagrange multiplier for $\Sigma\, q_y = 1$

At the minimum, a class weight only multiplies the predicted odds by w_B / w_H.

If the search scores plain CE

Plain CE is minimised by q = p, a strictly proper score, so the best weight is 1.

If the search scores the code's validation loss

The validation loss carries the same weight, so a lower w_B scores better for identical predictions.

2 The objective must be chosen first

Balanced CE = ½ (benign mean CE + high-grade mean CE) weighs both classes equally. With $w_B / w_H = n_H / n_B \approx 35 / 21 = 1.67$, training minimises the same criterion that the search scores.

Base code: trainer.py L69, L322 (validation uses the same weighted loss) · optuna_training.py L243, L500 · config.py L38
Gneiting & Raftery 2007 (log score is strictly proper) · n_H, n_B: high-grade and benign cases in an inner training set

Budget from pilot runs

Training and validation loss of one inner model (pilot)



The loss is class-weighted CE with the config weights (benign 2.53). The best epoch is 4, and early stopping ends the run at 12.

What I measured

One inner model takes 10–46 min (50–140 s per epoch).

A 19-case fold gives each balanced-CE score a standard error of 0.10.

Four seeds move the best validation loss with an SD of 0.07.

3 Trials closer than about 0.1 cannot be reliably ranked

With the same hyperparameters, two trials still differ by SD ≈ √2 × 0.07 / √4 ≈ 0.05 from seeds alone.

Budget

40 trials × 4 inner models × 5 outer folds = 800 runs, or 140–610 node-hours. One outer fold takes 7–31 hours with its 4 inner models in parallel.

The pilot is inner model 1 of outer fold A, trained on 56 cases and validated on 19 with config.py values; fold A is never used.
My code: Code2_NestedCVDesign_06Nov_Junho

Sampler and stopping rule

Where 16 startup points fall on one hyperparameter (16 equal bins)



Scrambled Sobol puts one point in every bin, on each hyperparameter's own scale. Random points leave 5.7 of 16 bins empty on average.
16 startup points are 3.2 per tuned dimension (d = 5) and 40% of the 40-trial budget.

TPE proposes where ℓ(x) / g(x) is high

$$EI(x) \propto \left(\gamma + (1 - \gamma)\, g(x) / \ell(x) \right)^{-1}$$

ℓ is fitted to the good trials and g to the rest.

4 Optuna's default good set is 10% of the trials, only 2 after 16, so one lucky trial carries a third of ℓ(x).

I set the good set to max(6, 15% of trials), the d + 1 = 6 minimum that BOHB uses.

A single stage is used, since trials within the noise floor cannot be ranked and a second stage would zoom in on noise.

The search stops at 40 trials, or after trial 30 if the best score gains less than 0.05 in 10 trials.

Pruning is off, since the pilot's validation loss moved by up to 0.8 between neighbouring epochs.

A range is widened and the search rerun when its pick lands in an end bin.

optuna/samplers/_tpe/sampler.py L53–54, parzen_estimator.py L59–69 (prior weight 1) · the random row is one draw
Bergstra et al. 2011 (TPE, γ = 0.15) · Falkner et al. 2018 (BOHB, one multidimensional density) · Owen 1998 (scrambled Sobol)

Final design

Overview › Tune › Scales › Objective › Budget › Sampler › **Summary**

Priority	Hyperparameter	Range	Scale	Why
1	**Learning rate η**	10^{-5} – 10^{-3}	log	Sets the size of every step
2	**Weight decay**	1% – 63% by epoch 4	log	**1** The old range barely acts
3	**Classifier dropout**	0 – 0.5	linear	Zero is valid; noise stays below signal
4	**Entropy weight λ**	10^{-3} – 1	log	The useful λ spans three orders
5	**Projector dropout**	0 – 0.5	linear	Zero is valid; noise stays below signal
–	**Class weight w_B / w_H**	n_H / n_B ≈ 1.67	fixed	**2** Set by the objective

Each trial scores balanced CE averaged over 4 inner models.

Sampling starts with 16 Sobol points and continues with multivariate TPE.

3 **The budget** is 800 runs in one stage, stopping at 40 trials or on a plateau after 30.

4 **The good set** is max(6, 15% of trials).

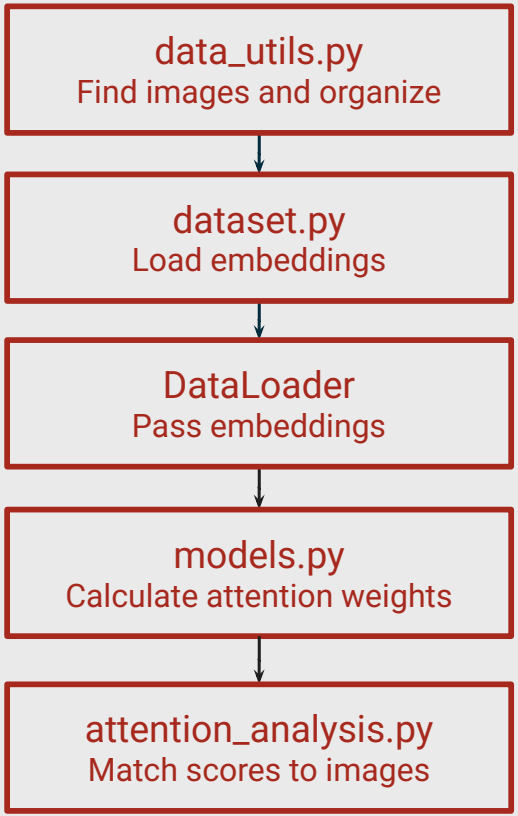
Next, outer fold A (160 runs) will show where the picks land and how much TPE improves on the Sobol start, followed by the other four folds.

Every number on these slides is recomputed by my code from the data, the pilot logs and the installed libraries
My code: Code2_NestedCVDesign_06Nov_Junho

Pairing Patches

Analyst 1, Pt. 1

Problem



Data preparation uses alphabetical sorting, attention_analysis.py uses numerical sorting

Model Order	Visualization Order
patch_1 → 0.10	patch_1 → 0.10
patch_10 → 0.20	patch_2 → 0.20 ✗
patch_11 → 0.30	patch_3 → 0.30 ✗
patch_2 → 0.30	patch_10 → 0.30 ✗
patch_3 → 0.10	patch_11 → 0.10 ✗

Solution

data_utils.py
Find images and organize

↓

dataset.py
Load embeddings and filenames

↓

DataLoader
Pass embeddings and filenames

↓

models.py
Calculate attention weights

↓

attention_analysis.py
Match scores to images using filenames

Model Order	Visualization Order
patch_1 → 0.10	patch_1 → 0.10 ☺
patch_10 → 0.20	patch_2 → 0.30 ☺
patch_11 → 0.30	patch_3 → 0.10 ☺
patch_2 → 0.30	patch_10 → 0.20 ☺
patch_3 → 0.10	patch_11 → 0.30 ☺

Preserve the Filenames

dataset.py

Include the filename lists

- Save each embedding and filename together
- Return tensors and filenames

_load_slice_tensor()

```
vecs.append(v)
filenames.append(os.path.basename(p))
```

```
return torch.stack(vecs, dim=0), filenames
```

Embeddings	Filenames
v_1	patch_1.png
v_{10}	patch_10.png
v_{11}	patch_11.png
v_2	patch_2.png
v_3	patch_3.png

Incorporate patch filenames into case items

__get_item__()

- Include the filenames lists for every slice within each stain
- Return dictionaries including the filenames

```
tensors_for_stain.append(sl_tensor)
filenames_for_stain.append(sl_filenames)
```

```
return {"case_id": case_id, "stain_slices": stain_slices,
"patch_filenames": filenames, "label": label}
```

Code: https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation2/Code1_HierarchicalMIL_06Nov_team/dataset.py

Use the Preserved Filenames

model.py
Calculate attention scores

- Receive the embeddings and calculate scores in the order it was received in

Embeddings	Attention
v_1	0.10
v_{10}	0.20
v_{11}	0.30
v_2	0.30
v_3	0.10

attention_analysis.py
Map scores to images

- Retrieve filenames from DataLoader

analyze_attention_weights()

```
patch_filenames = case_data["patch_filenames"]
```

- Pair each score to filename

compute_effective_patch_attention()

```
for patch_idx, pw in enumerate(patch_w):
    records.append({
        "patch_filename": filenames[patch_idx],
        "patch_attn_weight": float(pw.item())})
```

Patch	Attention
patch_1.png	0.10
patch_10.png	0.20
patch_11.png	0.30
patch_2.png	0.30
patch_3.png	0.10

- Retrieve image for visualization

visualize_case_effective_patches()

```
patch_filename = entry["patch_filename"]
png_path = os.path.join(PATCH_PNG_DIR,
os.path.splitext(patch_filename)[0] + ".png")
```

Code: https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation2/Code1_HierarchicalMIL_06Nov_team/attention_analysis.py

Raising Errors on Fallbacks and Silent Load Failures

Potential Issues	Error Raised	File
Missing patch PNG	FileNotFoundError	attention_analysis.py
Missing .pt embedding*	FileNotFoundError	dataset.py
Unreadable PNG	Runtime Error	attention_analysis.py
Unreadable .pt embedding	Runtime Error/TypeError	dataset.py
Missing case label	KeyError	dataset.py
Slice has no patches	ValueError	dataset.py
Missing/inconsistent attentions	ValueError	attention_analysis.py

* Currently commented out since checked and the only missing .pt embeddings are patches with width and/or height < 32, see code for this commented out test case and CheckMissingEmbeddings.ipynb

Code: https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation2/Code1_HierarchicalMIL_06Nov_team/dataset.py, https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation2/Code1_HierarchicalMIL_06Nov_team/attention_analysis.py, https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation2/Code1_HierarchicalMIL_06Nov_team/CheckMissingEmbeddings.ipynb

Results and Checks

Pipeline

Data loading: split 2, 74,335 patches across 86 cases
Training: stopped at epoch 14, best checkpoint epoch 6
Attention analysis: results saved
Errors: completed without runtime exceptions or other errors (see log)
Note: additional output run using split 1

Test Results

Test loss: 0.5992
Test accuracy: 0.8333
Samples: 18

Check 1: Creating Artificial Patches

dataset.py

- Create 20 artificial patches and embeddings
- Load patches using StainBagCasePooledFeatureDataset(shuffling_patches=True)
- Check that filename and embeddings are matched and that shuffling occurred
    - Repeat over different shuffles

```
Check 1 passed
```

Check 2:

attention_analysis.py

- Create 3 artificial filenames (not in numerical order) and attention weights
- Run analysis using compute_effective_patch_attention()
- Check that the filenames match the attention weights

```
Check 2 passed
```

Output (split 2): https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation2/Code1_HierarchicalMIL_06Nov_team/outputsSplit2_7091437.out
Code (checks 1 and 2): https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation2/Code1_HierarchicalMIL_06Nov_team/CheckPatches.ipynb
Additional output (split 1):
https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation2/Code1_HierarchicalMIL_06Nov_team/outputsSplit1_7068448.out

Locating Patches on Slices

Analyst 1, Pt. 2

The Problem & How the Algorithm Works

Krish's Patch Finder

Finds keypoints

- SIFT scans slices and patch at various zoom levels, finds distinctive, high contrast portions corresponding to nuclei and cell clusters

Accounts for patch rotations

- Each keypoint's strongest gradient becomes "up"

Matches patches onto their slice

- Patch and slice keypoints are stored as vectors, compared, then assigned a confidence score
- Patch position and orientation are found, and patches are outlined onto their slices

Code Reference: [Krish's outlining script]

What we added

Script to create overlays for top identified patches

- Given a model run and specific case numbers, a new code script reads the top_5_effective_attention_per_case.csv, and maps the top 5 patches onto a slice

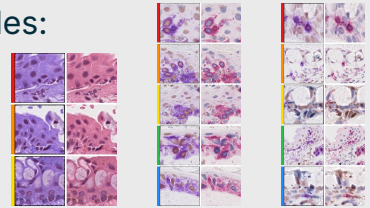
Produces a color ranked image per slice input

- Outline color corresponds to attention weight



And outputs a closeup of the patches next to their original slice regions to confirm match

- Examples:



Results Part 1: Melan A

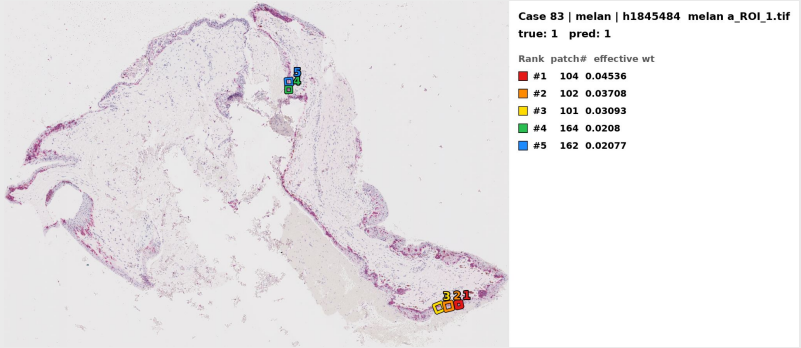
Patches Overlayed on Full Slice

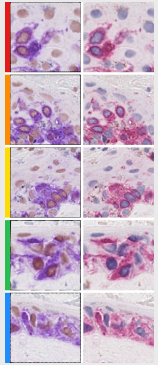
Matched Patch Close-ups

Things to note:

Case 83

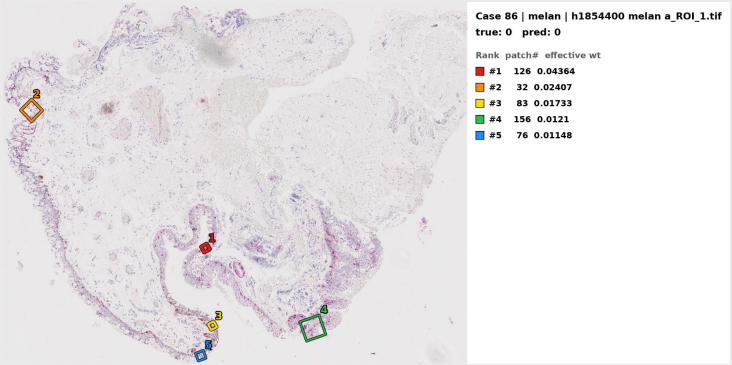
- Predicted high grade, and top attention patches neighbor each other
- We might expect malignancies to be near each other

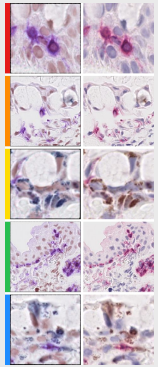




Case 86

- Predicted benign, top attention patches dispersed across slice
- This is also good benign cases must have no malignancies in any area of the sample





Code Reference: Github

Output Reference: Images

Results Part 2: H&E

Things to note:

Case 3

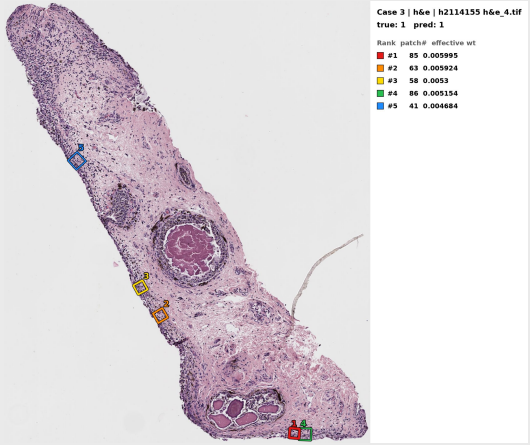
- Correct matches
- Relatively low, and roughly equal attention weights
- Unlike on last slide, this is a malignant slice with partly clustered patches

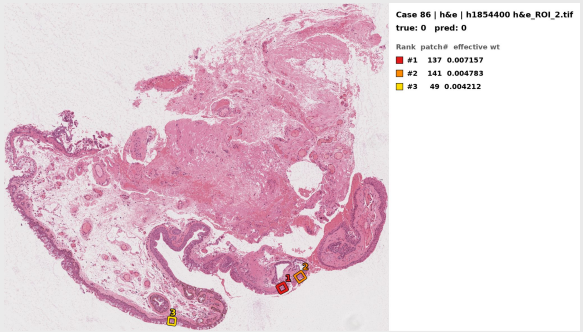
Case 86 (again)

- Correct matches
- Only 3 of top 5 patches were on this H&E slice
- Color differences between patches and slices particularly visible here

Code Reference: Github

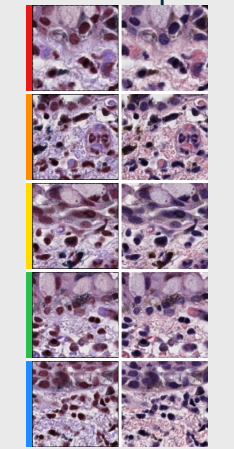
Patches Overlayed on Full Slice

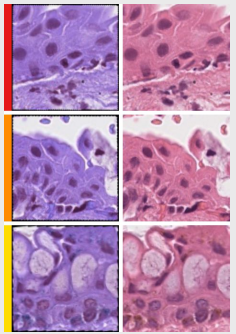




Output Reference: Images

Matched Patch Close-ups





Results Part 3: Sox10

Things to note:

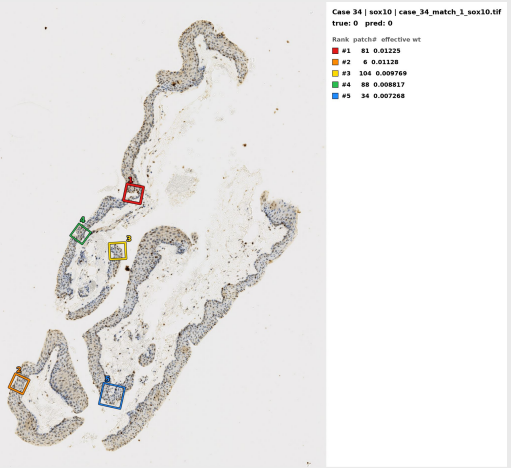
Case 34

- Moderate weights
- Handles slice separated into several pieces well

Case 90

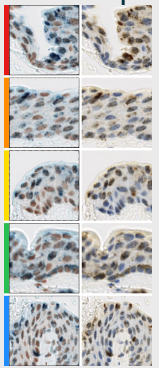
- Very high top attention weight

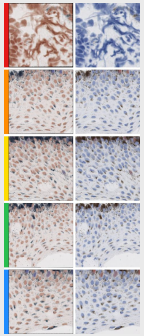
Patches Overlayed on Full Slice





Matched Patch Close-ups





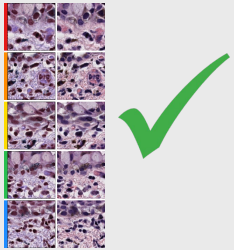
Code Reference: [Github]

Output Reference: [Images]

Results of Current Implementation

What works well

- All top patches tested were located correctly



- Runs relatively quickly, in a matter of minutes for several slices
- Handles rotations and color inconsistencies

What doesn't work well & limitations

Not every slice gets 5 boxes

- Model top patches CSV keeps only 5% of patches across all stains, so there won't always be 5 patches shown per slice

Low-contrast patches may not match

- SIFT relies on distinctive features like nuclei and faint/empty patches give few key points and lower confidence
- This will be important to properly characterize low attention patches, and determine if our model handles them well

Manual setup

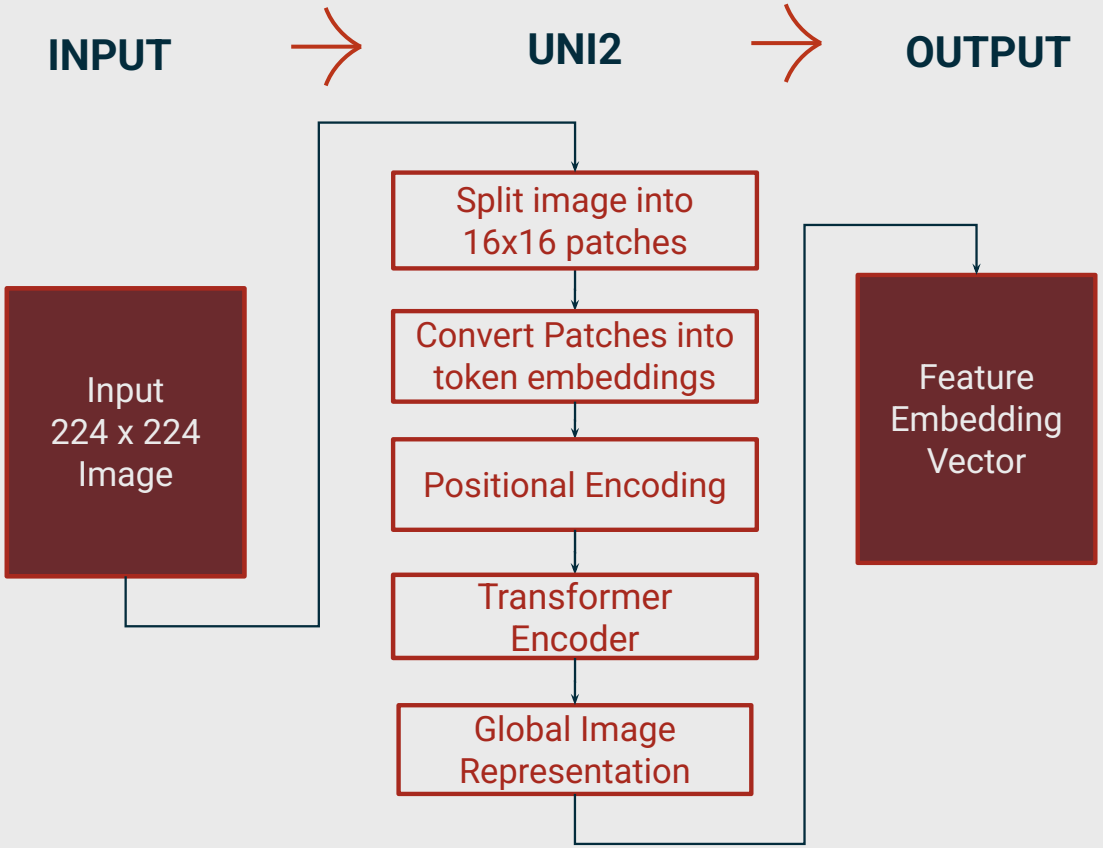
- .tif files are uploaded by hand, next steps include generalizing to any case

Sarah, Ashley

Differences Between UNI2 vs KimiaNet

Consultant

Overview of UNI2



- Input of a 224x224 image
- Images split into 16x16 patches
- Output of 1x1536 Feature Embedding Vector
- Kimia Net: uses features maps and adaptive pooling
- UNI2: uses tokens sequences and transformer attention

Difference 1: Why Training Image Size Matters

Training image size affects the **scale of tissue patterns** each model is exposed to during training and learns

Field of View: ***how much tissue the model sees***

- At the same magnification, KimiaNet's training images cover approximately 4.46× more tissue in each dimension and **20× more tissue area** than UNI2.
- KimiaNet's larger image can contain more cells, more epithelial layers, and more surrounding tissue structures at once.

Spatial Context: ***how well the model can understand relationships***

- Smaller training images emphasize **localized tissue regions**.
- Larger training images expose the model to more cells and broader tissue organization at once → more opportunity to learn **relationships**.

https://github.com/mahmoodlab/UNI
https://www.nature.com/articles/s41591-024-02857-3#data-availability
https://www.sciencedirect.com/science/article/pii/S1361841521000785

Difference 1: UNI2 vs. KimiaNet

Larger training images provide **context**; smaller images provide more **localized** views.

UNI2

Strengths

- May encourage learning of cellular morphology and smaller-scale tissue patterns.
- A localized abnormality can make up a larger proportion of an individual training example.

Weaknesses

- Less tissue context per training example.
- Less direct exposure to relationships spanning a large structure.

KimiaNet

Strengths

- More tissue and cells represented in each training example.
- Greater exposure to broader tissue architecture and relationships across epithelial layers.
- May better capture patterns that depend on the distribution or organization of cells.

Weaknesses

- Localized abnormalities may be less prominent.
- More surrounding tissue may introduce irrelevant variation.

https://github.com/mahmoodlab/UNI
https://www.nature.com/articles/s41591-024-02857-3#data-availability
https://www.sciencedirect.com/science/article/pii/S1361841521000785

Difference 1: Effects on our Project

Performance depends on what distinguishes benign from high-grade tissue

Example A: Melanocyte distribution across epithelial layers

- KimiaNet may have better performance
- Its larger training examples may provide more exposure to the relationship between melanocyte location and the overall epithelial structure.
- UNI2 may capture local melanocyte features but have less training-example context for the full vertical distribution.

Example B: A small, localized diagnostic feature

- UNI2 may have better performance
- Its smaller training examples may provide more focused exposure to localized morphology, with less surrounding tissue competing for representation.

https://link.springer.com/article/10.1186/s12859-018-2184-4?

Difference 2: Variable Sized Images with KimiaNet

KimiaNet (CNN)

- Patches are passed as their native size into KimiaNet
- If patch smaller than 32x32 than skipped
- Dynamically resize with Adaptive Average Pooling
- Output Fixed Embedded Feature Vector

PROS

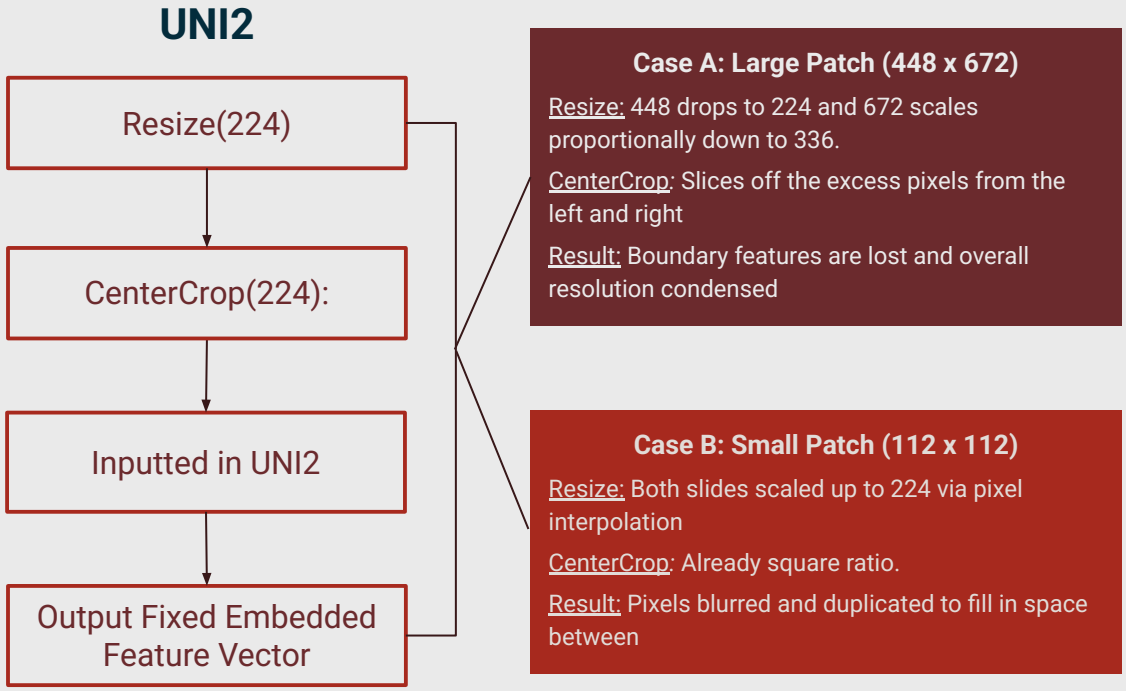
- Pixel Integrity
- Retains Spatial Data

CONS

- Scale Drift
- Non Uniformity
- Computational Time

https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation1_base_code/precompute_pooled_features.py
https://medium.com/@caring_smitten_gerbil_914/demystifying-nn-adaptiveavgpool2d-in-pytorch-why-adaptive-pooling-matters-in-deep-learning-1f7b7b1cc9b0

Difference 2: Variable Sized Images with UNI2



https://github.com/mahmoodlab/UNI
https://www.geeksforgeeks.org/python/image-resizing-using-opencv-python/
https://albumentations.ai/explore/transform/CenterCrop/docs/

- UNI2 was trained on 224x224 images. Current pipeline transforms all inputs to match
- Resize():scale smaller side to 224 and other proportionally to maintain aspect ratio
- CenterCrop():cut a 224x224 center square and remove leftover boundaries, if any

Difference 2: Variable Sized Images with UNI2

Effects on our Project

1

Marginal Information Loss:
Features located at the outer edges of tissue crops are removed with center crop.

2

Downstream MIL Consequences
Assumes every patch shares the same scale but a tumor cell in Patch A looks tiny, while an identical cell in Patch B looks humongous.

3

Artificial Magnification Shift
UNI2 is sensitive to spatial resolution and magnification. The patch token represents a physically consistent area of tissue and will force patches to scale up with interpolation or down and lose resolution.

https://towardsdatascience.com/compression-in-the-imagenet-dataset-34c56d14d463/

Difference 2: Variable Sized Images

KimiaNet (CNN)	UNI2 (ViT-L)
Variable Image Size	Variable Image Size
Embedded Feature Vector	Embedded Feature Vector

```
transform = transforms.Compose(
 [
  transforms.Resize(224),
  transforms.CenterCrop(224),
  transforms.ToTensor(),
  transforms.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.
 ]
```

```
#!/usr/bin/env
"""
precompute_pool

Precompute per-

UPDATED BEHAVIO...
- No resizing is applied.
- Each patch is forwarded at its original spatial size.
- Patches smaller than 32x32 are skipped.
- Because patch sizes vary, features are computed one patch at a time.
```

- Uses Resize and Center Cropping

Resized to 224 × 224
↓
14 × 14 token grid
↓
196 tokens

https://github.com/mahmoodlab/UNI/blob/main/README_old.md#slide-benchmarks
https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation1_base_code/precompute_pooled_features.py